

Contents

1	DD08: Cron 実装 (運営者システム)	1
1.1	0. 文書情報	1
1.2	1. 対象範囲	1
1.3	2. 収録ロジック・対応章	1
1.4	3. 詳細設計本文	1
1.4.1	3.1 Cron Triggers UTC cron 式表 (全 cron)	2
1.4.2	3.2 各 cron の擬似コード	2
1.4.3	3.3 Cron 失敗時の通知ポリシー	4
1.4.4	3.4 Cron UTC 一覧 (付録 L 同等)	5
1.4.5	3.5 wrangler.toml 設定例	5
1.4.6	3.6 cron 実装ガイドライン	5
1.5	4. 関連設計	5
1.6	5. テスト観点	6
1.6.1	5.1 ユニットテスト	6
1.6.2	5.2 結合テスト (Miniflare)	6
1.6.3	5.3 E2E テスト	6
1.6.4	5.4 受入条件マッピング	6
1.7	6. 未確定事項・確認事項	6

1 DD08: Cron 実装 (運営者システム)

1.1 0. 文書情報

項目	内容
文書名	DD08: Cron 実装 (運営者システム)
詳細設計 ID	DD08
対象システム	FAQ AI ウィジェット SaaS / 運営者システム
関連機能 ID	FR-303, AC-042, TH-10, D-04, D-05, D-08, D-09, D-11
作成日	2026-05-17
版数	v1.0
ステータス	承認済

1.2 1. 対象範囲

種別	ID	名称
機能	FR-303	月次請求確定 cron
受入条件	AC-042	月次請求 cron 幕等
Worker	MonthlyBillingCronWorker	月初請求書発行
Worker	AuditChainVerifierWorker	日次ハッシュチェーン検証
Worker	AnnouncementSchedulerWorker	1分ポーリング配信予約
Worker	DLQAutoBackoffWorker	5分指数 BO + 1h で dlq_manual_replay 遷移
Worker	RetentionPurgeWorker	日次 3 区分物理削除
Worker	R2AuditArchiveWorker	年次 R2 圧縮アーカイブ
Worker	OperatorNotifyAggregatorWorker	1分 10分集約処理

1.3 2. 収録ロジック・対応章

元章	元タイトル	概要
§ 14 全文	Cron 実装詳細 (★TH-10)	UTC cron 式表 + 擬似コード + 失敗時通知ポリシー
付録 L	Cron UTC 一覧	1 ページ集約

1.4 3. 詳細設計本文

基本設計 § 6.5 / § 6.2.7 / § 10.4 + D-04 / D-05 / D-09 / D-11 を物理化し、UTC cron 式を確定する。

1.4.1 3.1 Cron Triggers UTC cron 式表 (全 cron)

前提: Cloudflare Cron Triggers は標準 cron 構文(分・時・日・月・曜日)のみをサポートし、L(last day of month)・W・#等の拡張構文は使用できない。月末日付や JST の月初判定は **Worker 内のロジック**で行う(後述 § 3.1.1 ガード)。JST = UTC + 9 時間。

Worker	JST 想定	UTC cron 式	UTC 換算	Worker 内ガード	冪等性キー	リトライ	関連 D
MonthlyBillingCronWorker	月初 02:00 JST	0 17 * * * (毎日 17:00 UTC)	翌日 02:00 JST	if (jstNow().day !== 1) return;	(owner_account_id, billing_year_month) UNIQUE	3 回	D-11
AuditChainVerifierWorker	毎日 02:00 JST	0 17 * * *	翌日 02:00 JST	-	(verify_date)	-	D-04
AnnouncementSchedulerWorker	スケジューリング	* * * * *	毎分	-	-	-	D-09
DLQAutoBackoffWorker	5分スケジューリング	* / 5 * * * *	5 分間隔	-	(event_id, attempt)	指数 BO 3 回	-
RetentionPurgeWorker	毎日 03:00 JST	0 18 * * *	翌日 03:00 JST	-	(purge_date, retention_class)	-	D-08
R2AuditArchiveWorker	毎夜 12/31 04:00 JST	0 19 30 12 *	UTC 12/30 19:00 = JST 12/31 04:00	-	(archive_year)	-	-
OperatorNotificationAggregatorWorker	1分(10分集約処理)	* * * * *	毎分	-	KV TTL	-	D-19

1.4.1.1 3.1.1 Worker 内 JST 判定ガード (MonthlyBillingCronWorker) Cloudflare Cron Triggers は L(月末日)構文をサポートしないため、MonthlyBillingCronWorker は **毎日 17:00 UTC に起動し、JST 換算で当日が月初 (day === 1) である場合のみ処理を進める**：

```
function shouldRun(): boolean {
  const utcNow = new Date(); // Worker 起動時刻 (UTC)
  const jstNow = new Date(utcNow.getTime() + 9 * 60 * 60 * 1000);
  return jstNow.getUTCDate() === 1; // JST の日付が 1 日のとき true
}
```

```
if (!shouldRun()) {
  log.info("monthly-billing: skipped (jstDay !== 1)");
  return;
}
```

// 以降が §3.2.1 の処理

ガード判定自体は毎日実行されるが、**shouldRun()=false** で即 return するため副作用は発生しない。閏年・夏時間影響を受けず、確定的に「JST の月初日に 1 回だけ」実行される。

1.4.1.2 3.1.2 月末日のずれ対策 R2AuditArchiveWorker は **0 19 30 12 ***(UTC 12/30 19:00 = JST 12/31 04:00)を採用。JST の意図時刻を起点に UTC 計算した結果で、L 構文に依存しない。

UTC 換算の検算ルール(本書改訂時の必須チェック):

JST 想定	UTC 換算	cron 式	検算
03:00 JST	前日 18:00 UTC	0 18 <day-1> <month> *	翌日 03:00 JST に発火
02:00 JST	前日 17:00 UTC	0 17 <day-1> <month> *	翌日 02:00 JST に発火
04:00 JST	前日 19:00 UTC	0 19 <day-1> <month> *	翌日 04:00 JST に発火
12/31 04:00 JST	12/30 19:00 UTC	0 19 30 12 *	12/31 04:00 JST に発火

1.4.2 3.2 各 cron の擬似コード

1.4.2.1 3.2.1 MonthlyBillingCronWorker(D-11)

```

function monthlyBilling():
    target_month = (now - 1 day).format("YYYY-MM") // 月初実行時に前月を対象
    audit_logs(billing.cron.run, 7y, payload={month: target_month, started_at: now})

    owners = D1.query("SELECT id FROM accounts WHERE is_owner = 1 AND contract_status='active'")
    success = 0; failed = 0
    for owner in owners:
        try:
            usage = computeUsage(owner.id, target_month)
            amount = computeAmount(usage)
            existing = D1.query("SELECT id FROM billing_invoices WHERE owner_account_id=? AND b
            if existing:
                continue // 零等
            invoiceId = ULID()
            D1.exec("INSERT INTO billing_invoices (id, owner_account_id, billing_year_month, am
            stripeInvoice = stripe.invoiceItems.create({customer: owner.stripe_customer_id, amo
            stripeInv = stripe.invoices.create({customer: owner.stripe_customer_id, auto_advanc
            D1.exec("UPDATE billing_invoices SET stripe_invoice_id=? WHERE id=?", stripeInv.id,
            audit_logs(billing.invoice.issued, 7y, payload={invoice_id: invoiceId})
            sendMail(owner, FR-148)
            sendInbox(owner, FR-187)
            success++
        catch e:
            failed++
            if attempt >= 3:
                notifyOperator(high, "Monthly billing failed for owner=${owner.id}")

    audit_logs(billing.cron.run, 7y, payload={summary: {success, failed}})

```

1.4.2.2 3.2.2 AuditChainVerifierWorker(D-04)

```

function auditChainVerify():
    verify_date = today
    runId = ULID()
    audit_logs(audit.chain.verify.start, 5y, payload={verify_date, run_id: runId})

    prev_hash = "0" * 64
    mismatch_count = 0
    cursor = null
    do:
        rows = D1.query("SELECT id, prev_hash, record_hash, ... FROM audit_logs WHERE id > ? OR
        for r in rows:
            expected = sha256(prev_hash + canonical_json(r.{actor_id, action, ..., retention_cl
            if expected != r.record_hash:
                audit_logs(audit.chain.verify.fail, 5y, payload={row_id: r.id, expected, actual
                notifyOperator(high, "Audit chain mismatch at row=${r.id}")
                mismatch_count++
            prev_hash = r.record_hash
            cursor = r.id
    while rows.length > 0

    audit_logs(audit.chain.verify, 5y, payload={verify_date, run_id: runId, mismatch_count, tot

```

1.4.2.3 3.2.3 AnnouncementSchedulerWorker(D-09)

```

function announcementSchedule():
    candidates = D1.query("SELECT id FROM announcement_drafts WHERE state='scheduled' AND sched
    for c in candidates:
        D1.exec("UPDATE announcement_drafts SET state='sending' WHERE id=? AND state='scheduled'
        try:
            result = callMainIF(7, {announcementId: c.id, ...})
            if result.ok:

```

```

        D1.exec("UPDATE announcement_drafts SET state='sent', sent_at=? WHERE id=?", no
        audit_logs(announcement.send, 5y, payload={announcement_id: c.id, recipients: r
    else:
        handleRetry(c.id)
catch e:
    handleRetry(c.id)

```

1.4.2.4 3.2.4 DLQAutoBackoffWorker

```

function dlqAutoBackoff():
    candidates = D1.query("SELECT * FROM webhook_events WHERE state='failed' AND next_retry_at
    for e in candidates:
        try:
            payload = R2.get(`dlq-stripe-events/${e.event_id}.json`)
            result = callMainIF(10, payload)
            if result.ok:
                D1.exec("UPDATE webhook_events SET state='succeeded', last_transition_at=? WHERE
                D1.exec("INSERT INTO dlq_replay_log (id, event_id, replay_type, attempted_at, r
                audit_logs(stripe.event.processed, 7y)
            else:
                attempt = e.attempt_count + 1
                backoffMs = [1*60_000, 4*60_000, 16*60_000][attempt - 1]
                D1.exec("UPDATE webhook_events SET attempt_count=?, next_retry_at=?, last_trans
                if attempt >= 3:
                    D1.exec("UPDATE webhook_events SET state='dlq_manual_replay' WHERE event_id
                    notifyOperator(high, "Webhook DLQ stuck: ${e.event_id}")

        // 30 日経過 archived
        D1.exec("UPDATE webhook_events SET state='dlq_archived' WHERE state='dlq_manual_replay' AND

```

1.4.2.5 3.2.5 RetentionPurgeWorker(D-08)

```

function retentionPurge():
    today = today
    runId = ULID()
    audit_logs(retention.purge.run.start, 5y, payload={run_id: runId, date: today})

    for cls, days in [('1y', 365), ('5y', 365*5), ('7y', 365*7)]:
        cutoff = today - days
        deletedCount = D1.exec("DELETE FROM audit_logs WHERE retention_class=? AND occurred_at
        audit_logs(retention.purge.run, 5y, payload={run_id: runId, retention_class: cls, delet

    // 通知ログ・エラーログも個別保持
    D1.exec("DELETE FROM inbox_messages WHERE created_at < ?", today - 365)

```

1.4.2.6 3.2.6 R2AuditArchiveWorker

```

function r2AuditArchive():
    year = now.year - 1
    for cls in ['5y', '7y']:
        for month in 1..12:
            rows = D1.query("SELECT * FROM audit_logs WHERE retention_class=? AND occurred_at B
            if rows.length == 0: continue
            jsonl = rows.map(r => JSON.stringify(r)).join('\n')
            gzipped = gzip(jsonl)
            R2.put(`audit-archive/${cls}/${year}/${month}.tar.gz`, gzipped)
        audit_logs(retention.archive.run, 5y, payload={archive_year: year})

```

1.4.3 3.3 Cron 失敗時の通知ポリシー

Worker	連続失敗回数	通知
MonthlyBillingCronWorker	3 回	運営者 high
AuditChainVerifierWorker	1 回 (不一致検出)	運営者 high
AnnouncementSchedulerWorker	5 回 (同一 ID)	運営者 normal、対象 announcement を failed 遷移
DLQAutoBackoffWorker	3 回 / event	運営者 high(dlq_manual_replay 遷移時)

1.4.4 3.4 Cron UTC 一覧 (付録 L 同等)

Worker	UTC cron 式	JST 想定時刻	用途
MonthlyBillingCronWorker	0 17 * * * (毎日 17:00 UTC、Worker 内で JST 月初 1 日のみ実行)	翌月 02:00 JST	月次請求
AuditChainVerifierWorker	0 17 * * *	02:00 JST	ハッシュチェーン検証
AnnouncementSchedulerWorker	* * * * *	毎分	お知らせ予約配信
DLQAutoBackoffWorker	* / 5 * * * *	5 分間隔	DLQ 自動 BO
RetentionPurgeWorker	0 18 * * *	03:00 JST	保持期間別物理削除
R2AuditArchiveWorker	0 19 31 12 *	12/31 04:00 JST	年次 R2 アーカイブ
OperatorNotifyAggregatorWorker	* * * * *	毎分	FR-211 10 分集約処理

UTC ☒ JST 変換: JST = UTC + 9 時間。Cloudflare Cron Triggers は **L**(last day of month) をサポートしないため、月初・月末判定は Worker 内ガードで行う。

1.4.5 3.5 wrangler.toml 設定例

MonthlyBillingCronWorker の例:

```
name = "monthly-billing-cron"
main = "src/scheduler/monthly-billing.ts"
compatibility_date = "2026-01-01"
```

[triggers]

```
crons = ["0 17 * * *"] # 毎日 17:00 UTC、Worker 内で JST 月初 1 日のみ実行
```

[[d1_databases]]

```
binding = "DB"
database_name = "admin_db"
database_id = "<env-specific>"
```

[vars]

```
TIME_ZONE = "Asia/Tokyo"
```

1.4.6 3.6 cron 実装ガイドライン

項目	ガイドライン
冪等性	必ず冪等性キー (UNIQUE 制約またはアプリ層チェック) を持つ
実行時間制限	Cloudflare Workers の CPU 時間制限 (30 秒) に注意。大量処理はカーソルページング + 別 invocation へ分割
エラーハンドリング	try/catch で例外を吸収し、必ず audit_logs + notifyOperator を呼ぶ
再実行可能性	冪等性キー で重複実行を防ぐ。失敗ジョブは次回 cron tick で自動再試行 (Monthly-BillingCronWorker は 3 回まで)
監視	各 cron 開始時に <cron>.run.start、終了時に <cron>.run 監査ログを記録 (retention_class はジョブ性質に従う)

1.5 4. 関連設計

種別	参照先
要件	../01_ 要件定義/index.md
基本設計	../02_ 基本設計/index.md
課金・請求設計(正本)	../02_ 基本設計/10_ 課金・請求設計.md
関連 DD	DD03_ 監査ハッシュチェーン.md / DD04_Stripe_Webhook 一次受信.md / DD06_ お知らせ承認・配信.md
運用設計	../04_ 運用設計/index.md
将来対応	../05_future/index.md

1.6 5. テスト観点

1.6.1 5.1 ユニットテスト

- ・ JST 判定ガード (`shouldRun()`) の閏年 / 12 月 → 1 月 境界 / DST 影響なし
- ・ 指数バックオフ計算 (1m / 4m / 16m)
- ・ カーソルページング境界
- ・ 冪等性キー競合検出

1.6.2 5.2 結合テスト (Miniflare)

- ・ 月初 cron で 100 オーナー分の請求書発行 → `billing_invoices` UNIQUE 制約検証
- ・ ハッシュチェーン検証で 1000 行検証 → 改ざんなし
- ・ お知らせ scheduler が 1 分以内に `scheduled` → `sent` 完了
- ・ DLQ auto BO で `failed` → `succeeded` または `failed` → `dlq_manual_replay`
- ・ 保持期間 purge で `1y` / `5y` / `7y` の cutoff 計算

1.6.3 5.3 E2E テスト

- ・ 月次請求 cron の手動トリガ (`wrangler cron trigger`) で本番相当データ処理
- ・ ハッシュチェーン検証バッチの不一致シミュレーション

1.6.4 5.4 受入条件マッピング

AC	検証手段
AC-042(月次請求 cron 冪等)	統合テスト

1.7 6. 未確定事項・確認事項

確認事項 ID	確認内容	優先度	ステータス
-	v1.0 リリース時点で全項目確定済み	低	確認済